

Preprocessamento dei dati

Laboratorio di Intelligenza Artificiale

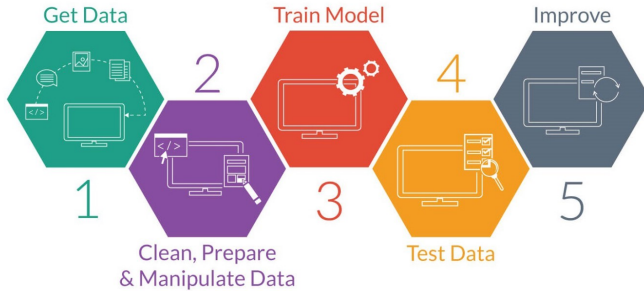
Vincenzo Bonnici

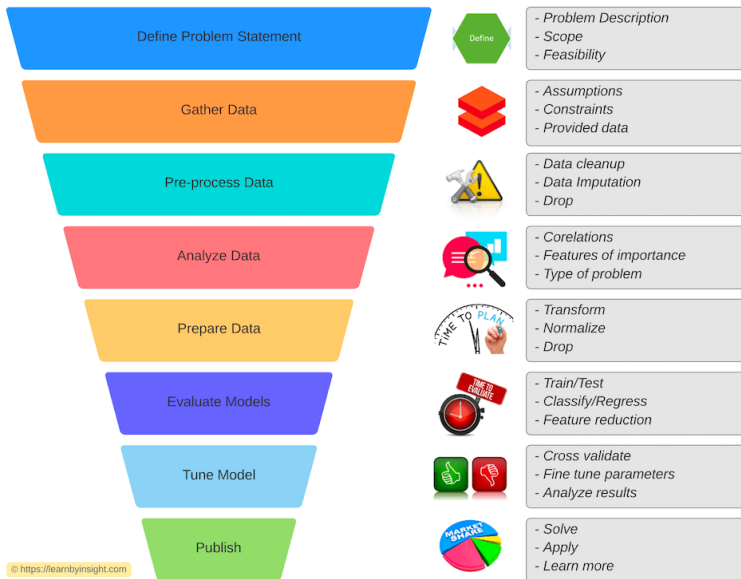
Corso di Laurea Magistrale in Scienze Informatiche

Dipartimento di Scienze Matematiche, Fisiche e Informatiche

Università degli Studi di Parma

2026-2027





- **data cleaning**: preprocessing dei dati allo scopo di ridurre rumore e trattare i valori mancanti
- **feature selection**: rimuovere i dati non rilevanti
- **data transformation**: generalizzare e normalizzare i dati

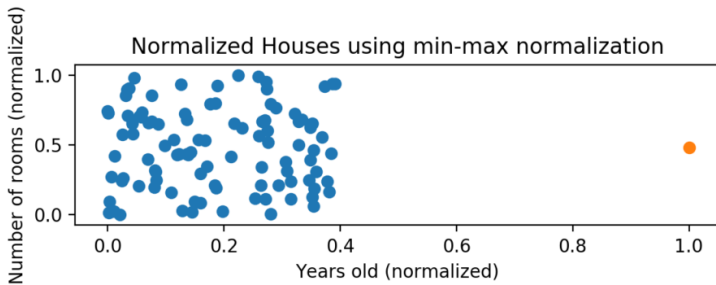
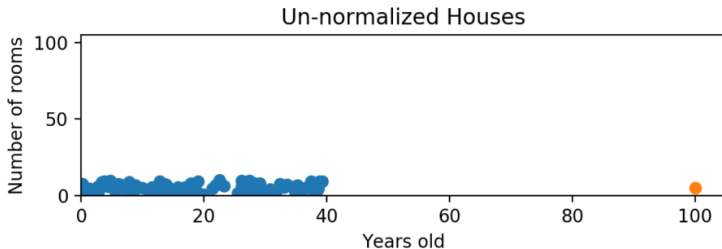
Tipi di dato:

- variabili **binarie** ($[0,1]$, presente/assente)
- **nominali** (rosso, giallo, blue green)
- **ordinali** (cattivo, buono, eccellente)
- **Interval-scaled** variables (Variabili a scala di intervallo): misurazioni continue di una scala approssimativamente lineare (peso, altezza, temperatura meteorologica)
- variabili di **rapporto** (ratio)
- tipi **misti**

L'**unità di misura** utilizzata può influenzare il analisi di clustering: passaggio da chilogrammi a libbre può dare un diverso raggruppamento struttura.

Per evitare la dipendenza dalla scelta di unità di misura, i dati dovrebbero essere **normalizzati o standardizzati** in modo da avere tutte un uguale peso/contributo.

Normalizzazione



Dato un insieme di dati $\mathbb{X} = \{x_1, \dots, x_n\}$, indichiamo con $\hat{x}_i$ il valore normalizzato di x_i .

Normalizzazione **0-max**: riscaliamo i valori numeri in modo da essere proiettati nel range $[0 \dots 1]$ con 0 pari allo zero assoluto

$$\hat{x}_i = \frac{x_i}{\max(\mathbb{X})}$$

Problema: possiamo avere **valori negativi** in $\mathbb{X}$.

Come soluzione possiamo riscalare i dati nel range $[-1 \dots 1]$

$$\hat{x}_i = \frac{x_i}{\max(\{|x_i| : x_i \in \mathbb{X}\})}$$

Normalizzazione **min-max**: riscaliamo i valori numeri in modo da essere proiettati nel range $[0 \dots 1]$ con 0 pari al minimo dei valori di input

$$\hat{x}_i = \frac{x_i - \min(\mathbb{X})}{\max(\mathbb{X}) - \min(\mathbb{X})}$$

Non abbiamo più il problema dei valori negativi, essendo ora $\min(\mathbb{X})$ mappato su 0 indipendentemente dal fatto che sia positivo o negativo.

Problema: è sensibile ai valori di $\mathbb{X}$. Ovvero, per diversi $\mathbb{X}$ e $\mathbb{X}'$ tale che $\min(\mathbb{X}) \neq \min(\mathbb{X}')$ o $\max(\mathbb{X}) \neq \max(\mathbb{X}')$, lo stesso valore x presente in entrambi è mappato su un valore diverso.

Normalizzazione **min-max sulla media**: riscaliamo i valori numeri in modo da essere proiettati nel range $[-1 \dots 1]$ con il valore $media(\mathbb{X})$ mappato su 0

$$\hat{x}_i = \frac{x_i - media(\mathbb{X})}{max(\mathbb{X}) - min(\mathbb{X})}$$

Deviazione media assoluta di un variabile f con n valori è definita come:

$$s_f = \frac{1}{n}(|x_{1f} - m_f| + |x_{2f} - m_f| + \cdots + |x_{nf} - m_f|)$$

dove $m_f = \frac{1}{n}(x_{1f} + x_{2f} + \cdots + x_{nf})$.

Lo **z-score**, ovvero la misura standardizzata che mappa i dati nell'intervallo $[-\infty \cdots + \infty]$. E' definita come:

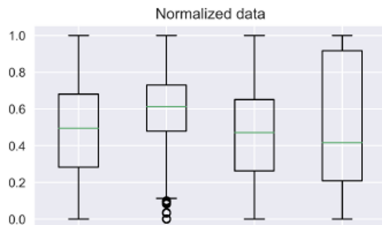
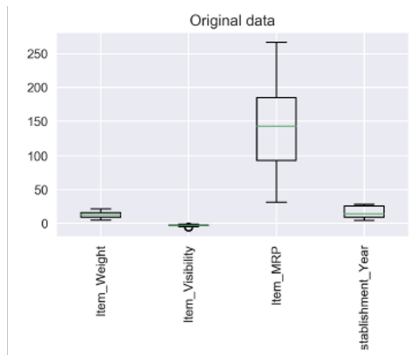
$$z_{if} = \frac{x_{if} - m_f}{s_f}$$

o più comunemente

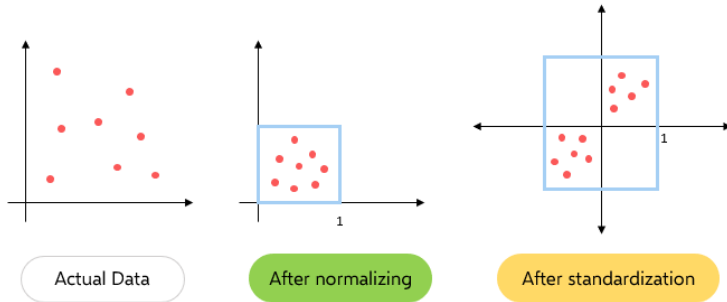
$$z_i = \frac{x_i - \mu}{\sigma}$$

con μ la media e σ la deviazione standard.

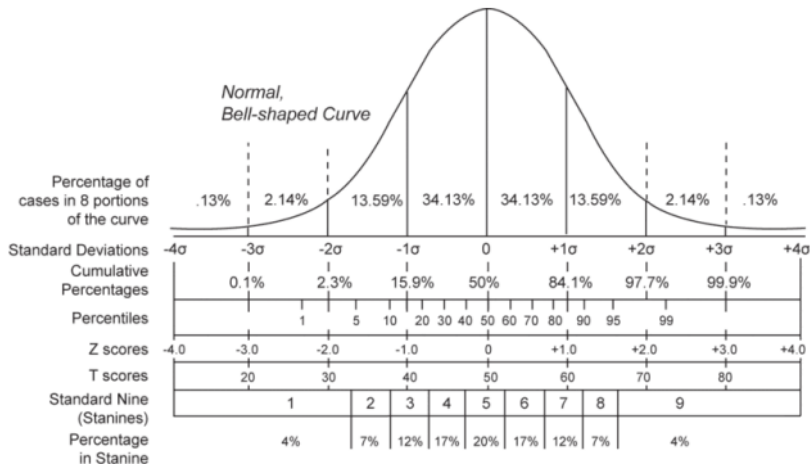
Normalizzare e standardizzare



Normalizzare e standardizzare



Normalizzare e standardizzare



La maledizione della dimensionalità , introdotta per la prima volta dal matematico Richard Bellman, indica che **il numero di campioni necessari per stimare una funzione arbitraria con un dato livello di accuratezza cresce esponenzialmente rispetto al numero di variabili di input (cioè dimensionalità) della funzione.**

$$\#funzioni \gg \#campioni$$

La maledizione della dimensionalità si riferisce alla situazione in cui i nostri dati hanno troppe caratteristiche, cioè l'algoritmo non sta imparando la funzione di destinazione richiesta ma invece viene catturato nel **vortice delle dimensioni**.

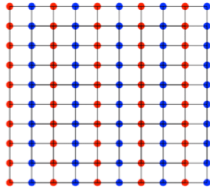
È simile a quando qualcuno cerca di spiegarti qualcosa in modo complicato, parlando di cose che non riguardano l'argomento in questione. La persona è completamente fuori tema con così tante **caratteristiche non importanti**.

Le funzionalità aggiunte agiscono come **rumore** nei dati, il che riduce la precisione del nostro modello.

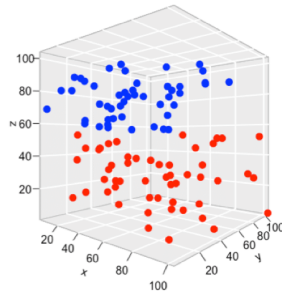
Curse of dimensionality



(A) 1-D

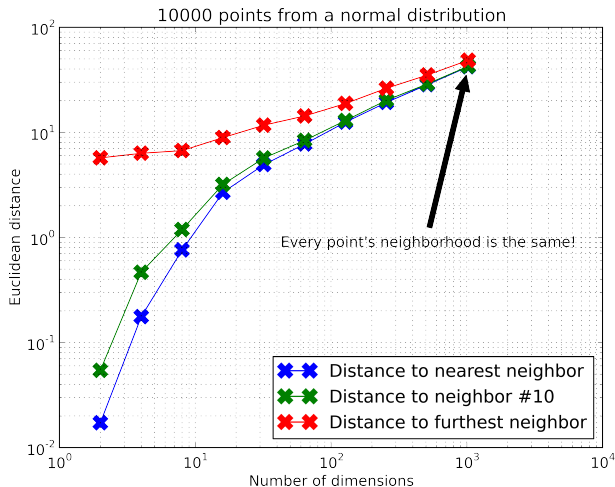


(B) 2-D



(C) 3-D

Curse of dimensionality

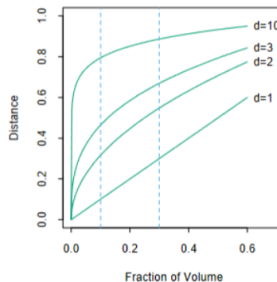
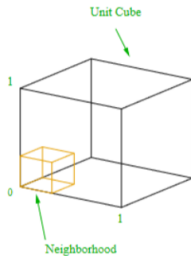


Curse of dimensionality

Let us consider the unit cube $[0, 1]^d$. The sample data is uniformly distributed within this cube. Then, the expected edge length of a cube that contains a fraction f of the data is $f^{\frac{1}{d}}$.

Proof: Since the data is uniformly distributed within the unit cube then the expected volume of a cube that contains a fraction f is f (of course $f \in [0, 1]$). Since the dimension is d , then the expected edge length of a cube is $f^{\frac{1}{d}}$.

This means that the size of the edge that you will need to encompass a certain fraction of the volume will increase drastically, specially at the beginning. i.e. the edge you need will become ridiculously large as D increases.



Curse of dimensionality e overfitting

L'overfitting si verifica quando **stimano relativamente pochi parametri nello spazio dimensionale superiore o se stimano molti parametri nello spazio dimensionale inferiore.**

Inoltre, la varianza della stima di un parametro aumenta se il numero di parametri da stimare aumenta.

Al contrario, la **distorsione della stima** e la dimensione dell'insieme di addestramento sono costanti, il che suggerisce che la qualità delle stime dei parametri diminuisce all'aumentare della dimensionalità a causa dell'aumento della varianza.

Se abbiamo più funzioni che osservazioni corriamo il rischio di **sovradimensionare** in modo massiccio il nostro modello, ciò generalmente si traduce in prestazioni terribili fuori campione.

La **crescita esponenziale dei dati** causa un'elevata scarsità nel set di dati, cioè il volume dello spazio rappresentato prolifera così tanto che i dati non riescono a mantenere la loro struttura.

Quando abbiamo **troppe caratteristiche**, le osservazioni diventano più **difficili da raggruppare**.

Troppe dimensioni fanno sì che **ogni osservazione nel set di dati appaia equidistante da tutte le altre**. E poiché il raggruppamento utilizza una distanza (ad esempio la distanza euclidea) per quantificare la somiglianza tra le osservazioni, questo è un grosso problema.

Ci viene in aiuto la percentuale di spazio totale riempito da un'ipersfera all'interno di un ipercubo

$$\frac{\pi^{\frac{n}{2}}}{2^n \Gamma(\frac{n}{2} + 1)}$$

La formula sopra generalizza la percentuale di spazio riempito da un'ipersfera all'interno di un ipercubo per tutte le dimensioni n , dove $\Gamma(n/2 + 1)$ è la funzione Gamma ($\Gamma(n) = (n - 1)!$).

Si può facilmente osservare che la quantità di cui sopra tende ad avvicinarsi a zero per valori più alti di n , il che suggerisce che sempre più dati iniziano a concentrarsi agli angoli (cioè, verso una singola dimensione) dell'ipercubo, il che non solo aumenta la scarsità ma rende inoltre obsoleta la funzione distanza.

Curse of dimensionality e potenza computazionale

Lavorare con i dati diventa più impegnativo con l'aumentare del numero di dimensioni. Come molte sfide nella scienza dei dati, questo si riduce alla **combinatoria**.

Con dati ad alta dimensione, semplicemente **non possiamo campionare in modo completo tutte le possibili combinazioni**, lasciando nell'oscurità vaste regioni dello spazio delle caratteristiche.

Potremmo anche non aver bisogno di sottoporre le nostre macchine a un lavoro così impegnativo. Avere molte dimensioni non garantisce che ogni dimensione sia particolarmente utile. La maggior parte delle volte potremmo misurare lo stesso pattern sottostante in molti modi diversi.

In generale, i valori aggiunti da molte dimensioni nei dati ad alta dimensione sono **ridondanti** per i nostri algoritmi.

"Come decidere il numero migliore di funzionalità per la nostra formazione?"

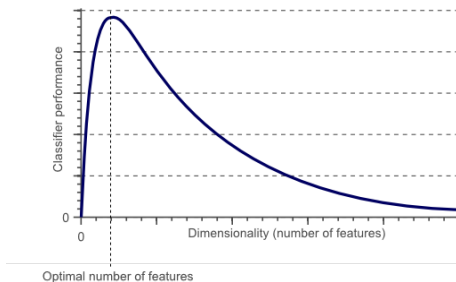
Sfortunatamente, nessuna regola fissa definisce quante funzionalità dovremmo usare in un problema di regressione / classificazione. Il numero magico dipende

- dalla **quantità di dati** di addestramento disponibili
- dalla **complessità dei confini decisionali**
- dal tipo di **classificatore** utilizzato

Ad esempio, se fosse disponibile un numero teorico di esempi di addestramento, non dovremmo affrontare questa maledizione. Potremmo semplicemente utilizzare un numero infinito di funzioni per ottenere la classificazione perfetta. Tuttavia, un numero infinito di funzionalità richiede un numero infinito di esempi di addestramento, eliminando l'utilità del mondo reale della nostra rete!

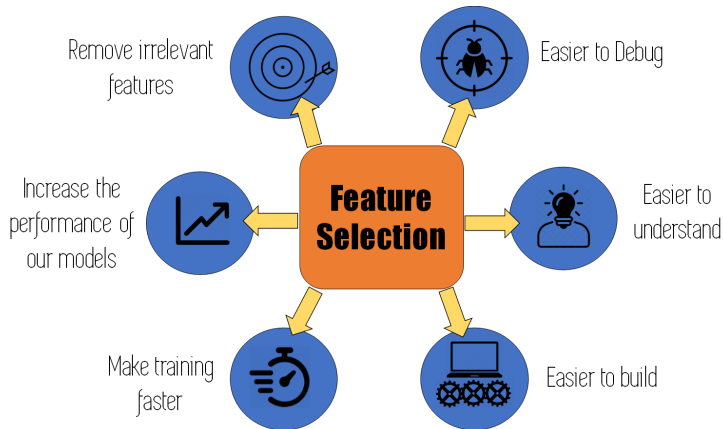
Curse of dimensionality: fenomeno di Hughes

Il fenomeno di Hughes mostra che all'**aumentare del numero di funzionalità, aumentano anche le prestazioni del classificatore fino a raggiungere il numero ottimale di funzionalità**. L'aggiunta di più funzionalità basate sulla stessa dimensione del set di addestramento ridurrà le prestazioni del classificatore.



La cosa più sconcertante è che **il numero di dati di allenamento necessari aumenta in modo esponenziale con ogni funzionalità aggiunta**.

Curse of dimensionality - soluzione 1: feature selection



Gli algoritmi di selezione delle caratteristiche sono classificati come supervisionati, che possono essere utilizzati per i dati etichettati; o non supervisionato, che può essere utilizzato per dati non etichettati.

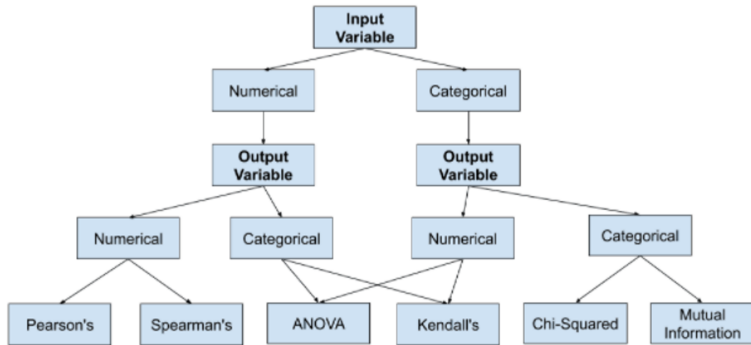
Le **tecniche non supervisionate** sono classificate come metodi di **filtro** (filter), metodi **wrapper**, metodi **incorporati** o metodi ibridi (embedded).

Metodi di filtro: i metodi di filtro **selezionano le funzionalità in base alle statistiche anziché alle prestazioni** di convalida incrociata della selezione delle funzionalità. Viene applicata una metrica selezionata per identificare gli **attributi irrilevanti** ed eseguire la **selezione ricorsiva** delle caratteristiche. I metodi di filtro sono **univariati**, cioè viene stabilita una classifica ordinata delle funzionalità per informare la selezione finale del sottoinsieme di funzionalità; o **multivariato**, che valuta la rilevanza delle caratteristiche nel loro insieme, individuando caratteristiche ridondanti e irrilevanti.

Metodi wrapper: considerano la **selezione di un insieme di caratteristiche come un problema di ricerca**, per cui la loro qualità viene valutata con la preparazione, la valutazione e il confronto di una combinazione di caratteristiche con altre combinazioni di caratteristiche. Questo metodo facilita l'individuazione di possibili **interazioni tra variabili**. I metodi wrapper si concentrano su sottoinsiemi di funzionalità che aiuteranno a migliorare la qualità dei risultati dell'algoritmo.

Metodi incorporati: **integrano l'algoritmo di apprendimento automatico di selezione** delle funzioni come parte dell'algoritmo di apprendimento, in cui la classificazione e la selezione delle funzioni vengono eseguite contemporaneamente. La selezione casuale delle caratteristiche della foresta, la selezione delle caratteristiche dell'albero decisionale e la selezione delle caratteristiche LASSO sono metodi incorporati comuni.

È comune utilizzare misure statistiche di tipo di **correlazione** tra variabili di input e output come base per la selezione delle funzioni di filtro.



Esercizi

Applichiamo feature selection e correlazione tra le feature ai dataset di scikit-learn.

- quali sono le feature con la varianza più alta?
- nel considerare la varianza ci conviene normalizzare?
- quale è la dipendenza tra le feature?
- possiamo calcolare una dipendenza tra una feature categoriale ed una numerica?

Curse of dimensionality - soluzione 2 : riduzione della dimensionalità

Il modo migliore per addestrare il nostro modello sui dati di addestramento limitati è **ridurre il numero di funzioni (o dimensioni)**. Un algoritmo in grado di farlo è noto come **Dimensionality Reduction**.

- L'algoritmo inizia individuando le **tendenze** sottostanti nelle nostre funzionalità, ovvero estraendo le caratteristiche dai dati (queste tendenze sottostanti nascoste nelle nostre funzionalità sono spesso chiamate funzionalità **latenti**).
- Successivamente, l'algoritmo stima e **riscrive** ogni altra caratteristica (proietta in un altro sistema) in termini di esposizione a queste caratteristiche latenti.
- Possiamo quindi classificare i punti dati utilizzando misure di distanza.

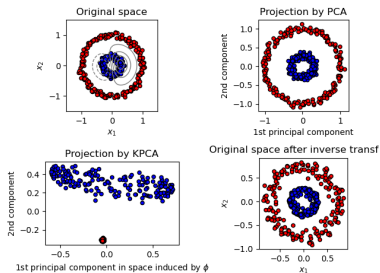
Le più note tecniche di riduzione di dimensionalità sono:

- **Principal Component Analysis** (PCA o KL transform): trasformazione non supervisionata che esegue un mapping lineare con l'obiettivo di preservare al massimo l'informazione dei pattern
- **Linear Discriminant Analysis** (LDA): il mapping è ancora lineare ma in questo caso è supervisionato. LDA privilegia le dimensioni che discriminano al meglio i pattern del TS
- ...continua...

- **t-distributed Stochastic Neighbor Embedding** (t-SNE): trasformazione non lineare e non supervisionata specificatamente ideata per ridurre dimensionalità a 2 o 3 dimensioni onde poter visualizzare dati multidimensionali
- **Uniform Manifold Approximation and Projection for Dimension Reduction** (UMAP): simile a t-SNE ma generalizza nel caso non lineare. Da alcune ipotesi sulla distribuzione dei dati è possibile modellare la varietà con una struttura topologica fuzzy.
- ...continua...

Riduzione della dimensionalità

- **Independent Component Analysis** (ICA): trasformazione lineare orientata a proiettare i dati su una base di componenti (statisticamente indipendenti)
- **Local Linear Embedding** (LLE): trasformazione non lineare che invece di calcolare un mapping "globale", considera relazioni tra gruppi di punti vicini
- **Kernel PCA**: simile a PCA ma più potente perché il mapping è non lineare. Utilizza un "trucco" simile a quello che permette di passare da SVM lineare a SVM non lineare



Anowar, F., Sadaoui, S., & Selim, B. (2021).

Conceptual and empirical comparison of dimensionality reduction algorithms (pca, kpca, lda, mds, svd, lle, isomap, le, ica, t-sne).

Computer Science Review, 40, 100378.

Questa classificazione divide i metodi in due classi:

- **random projection**: proietta i dati originali su un nuovo spazio utilizzando una matrice "random" (aleatoria), tale che la radice quadrata della somma dei quadrati di ogni elemento della colonna è 1. Preserva la proporzionalità delle distanze tra i dati originali.
- **manifold**: sono generalmente metodi non lineari, non supervisionati e basti su grafi. Vengono specificatamente adottati per mappare i dati in uno spazio a dimensionalità più bassa assumendo un campionamento denso dei dati e che questi risiedano su un "manifold" (spazio multiforme) da srotolare.

Riduzione della dimensionalità

Table 2
Conceptual Comparison of FEAs.

	Goal	Supervision	Linearity	Data Charac.	Iteration	Topology
PCA	Maximize variance	Unsuper.	Linear	Handle skewed data	No	Random Projection
LDA	Maximize class separation	Super.	Linear	Perform better with uniformly distributed data	No	Random Projection
MDS	Preserve Euclidean pairwise distances	Unsuper.	Nonlinear	Perform better with relational data	Yes	Manifold
KPCA	Linearly separate data	Unsuper.	Nonlinear	Perform better with nonlinear data	No	Manifold
SVD	Minimize reconstruction error	Unsuper.	Linear	Handle sparse data	Yes	Random Projection
LLE	Preserve local properties	Unsuper.	Nonlinear	Handle sparse and nonlinear data	Yes	Manifold
ISOMAP	Preserve geodesic pairwise distances	Unsuper.	Nonlinear	Handle sparse and noisy data	No	Manifold
LE	Preserve local distances	Unsuper.	Nonlinear	Handle noisy data	No	Manifold
ICA	Maximize statistical independence	Super.	Linear	Handle higher order statistics	Yes	Random Projection
t-SNE	Preserve local structure	Unsuper.	Nonlinear	Perform better with numeric data	Yes	Manifold

Table 3
Conceptual Comparison of FEAs (cont')

	Tuning of Parameters	Computational Complexity	Weakness	Transformation
PCA	No. of components	$O(d^2n + n^3)$	Limited to linear projection	Eigen decomposition
LDA	None	$O(d^2n)$, $n > d$; $O(d^3)$, $d > n$	Suffer from class singularity issue	Class separability by scatter matrices
MDS	No. of components and iterations	$O(n^3)$	Require high computation and memory to calculate dissimilarity matrix	Dissimilarity matrix by Euclidean distance
KPCA	No. of components, Kernel	$O(n^3)$	Require high training time	Kernel mapping
SVD	No of components and iterations	$O(d^2n + n^3)$	Not suitable for nonlinear data	SVD theorem
LLE	No. of components, iterations and nearest neighbors	$O(d \log(c)n \log(n))$ $+O(dnc^3)+$ $O(kn^2)$	Consume memory	Neighborhood graph, reconstruction weights
ISOMAP	No. of components and nearest neighbors	$O[d \log(c)n \log(n)] +$ $O[n^2(c + \log(n))] +$ $O(kn^2)$	Suffer from topological instability	Neighborhood graph, geodesic distance
LE	No. of components and nearest neighbors	$O(d \log(c)n \log(n))$ $+O(dnc^3)+$ $O(kn^2)$	May produce disconnected neighborhood graph	Neighborhood graph, weight update of edges
ICA	No. of components and iterations, Tolerance	$O[2di(d+1)n]$	Require high training time	Non-Gaussian distribution
t-SNE	No. of components and iterations	$O(n^2)$	Provide only 2 or 3 features	Conditional probabilities

PCA cerca di spiegare la struttura di correlazione di un insieme di variabili predittive, utilizzando un insieme più piccolo di **combinazioni lineari** di queste variabili. Queste combinazioni lineari sono chiamate **componenti**.

Significa che la **variabilità totale** di un insieme di dati prodotta dall'insieme completo di m **variabili** può essere **spiegata** da un insieme più piccolo di k **combinazioni lineari**. Ci sono quasi tante informazioni nelle componenti k quante ce ne sono nelle m variabili originali.

Dato un training set $TS = \{x_i \in \mathbb{R}^d, i = 1 \dots n\}$, siano:

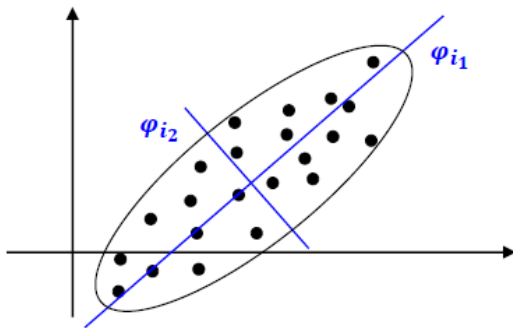
$$\bar{x} = \frac{1}{n} \sum_{i=1 \dots n} x_i ; \Sigma = \frac{1}{n-1} \sum_{i=1 \dots n} (x_i - \hat{x})(x_i - \hat{x})^t$$

il **vettore medio** $\bar{x} \in \mathbb{R}^d$ e la **matrice di covarianza** $\Sigma \in \mathbb{R}^{d \times d}$, allora per un dato k ($k < d$, $k < n$, $k > 0$) lo spazio k dimensionale $(S_{\bar{x}, \Phi_k})$ è univocamente definito dal vettore medio e dalla matrice di proiezione $\Phi_k \in \mathbb{R}^{d \times k}$ le cui colonne sono costituite dagli autovettori di Σ corrispondenti ai k più grandi autovalori:

$$\Phi_k = [\phi_{i_1}, \phi_{i_2} \dots \phi_{i_k}] \text{ con } \lambda_{i_1} \geq \lambda_{i_2} \dots \geq \lambda_{i_k} \dots \lambda_{i_d}$$

essendo ϕ_{i_r} autovettore di Σ corrispondente all'autovalore λ_{i_r} , $r = 1 \dots d$

ϕ_{i_1} indica la **direzione di maggior varianza** nel TS training set



I primi k autovettori sono detti **componenti principali**

PCA: proiezione e retroproiezione

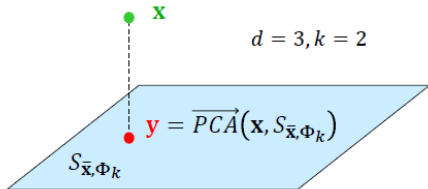
Proiezione: una volta determinato lo spazio PCA, la proiezione di un pattern su tale spazio è semplicemente la **proiezione geometrica** di un vettore sull'iperpiano che definisce lo spazio.

In realtà, la vera proiezione geometrica è un vettore che ha la stessa dimensionalità del vettore originale mentre in questo contesto indichiamo con **proiezione il vettore (ridotto) nello spazio PCA**.

Matematicamente questa operazione è eseguita come prodotto della **matrice di proiezione trasposta** per il pattern al quale è preventivamente **sottratta la media**

$$\overrightarrow{PCA}: \mathbb{R}^d \rightarrow \mathbb{R}^k$$

$$\overrightarrow{PCA}(\mathbf{x}, S_{\bar{\mathbf{x}}, \Phi_k}) = \Phi_k^t (\mathbf{x} - \bar{\mathbf{x}})$$



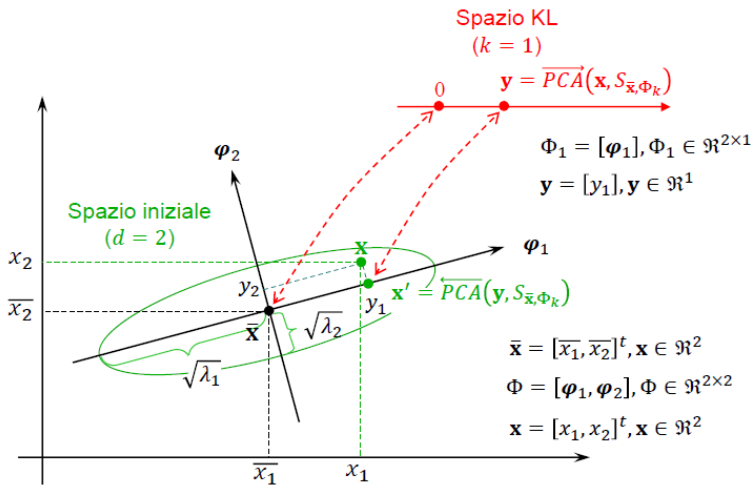
Retro proiezione: dato un vettore nello spazio PCA, la sua retro proiezione verso lo spazio originale si ottiene moltiplicando il vettore per la matrice di proiezione e sommando il vettore medio.

Questa trasformazione non sposta spazialmente il vettore, che giace ancora sullo spazio PCA ma opera un cambiamento di coordinate che ne permette la codifica in termini delle d componenti dello spazio originale

$$\overleftarrow{PCA}: \Re^k \rightarrow \Re^d$$

$$\overleftarrow{PCA}(\mathbf{y}, S_{\bar{\mathbf{x}}, \Phi_k}) = \Phi_k \mathbf{y} + \bar{\mathbf{x}}$$

PCA: esempio riduzione 2 $\mapsto$ 1



- L'**ellisse** rappresenta la **distribuzione dei punti** nel training set.
- ϕ_1 e ϕ_2 sono gli **autovettori** della matrice di covarianza.
- Gli **autovalori** λ_1 e λ_2 sono le **varianze della distribuzione lungo gli assi** ϕ_1 e ϕ_2 .
- y_1 e y_2 sono le **proiezioni** di x sugli assi ϕ_1 e ϕ_2
- Se λ_2 è piccolo, x può essere **approssimato** con x' (retroproiezione di y) **senza perdite significative** di informazione.
- Si può dimostrare che tra tutte le riduzioni di dimensionalità lineari PCA è quella che **preserva al massimo l'informazione** dei vettori originali.

PCA: scelta di k

Talvolta la scelta di k è obbligata ad esempio per la visualizzazione 2D o 3D dei dati.

Quando invece l'obiettivo è quello di scartare informazione inutile e dati correlati **mantenendo gran parte del contenuto informativo** si può scegliere k nel modo seguente.

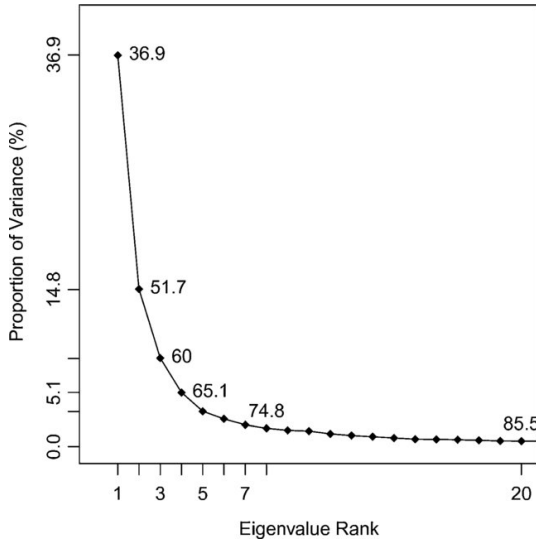
Fissata una percentuale t del contenuto informativo che si vuole preservare (es $t = 95\%$) si sceglie il minimo valore di k per cui la somma dei più grandi k autovalori rispetto alla somma di tutti gli autovalori è maggiore o uguale a t .

Considerando gli autovalori ordinati in ordine decrescente

$$k = \operatorname{argmin}_z \left[\frac{\sum_{i=1 \dots z} \lambda_i}{\sum_{i=1 \dots d} \lambda_i} \geq t \right]$$

Infatti, ricordando che gli autovalori denotano la varianza lungo i diversi assi, il rapporto nella formula indica la **varianza conservata** rispetto alla varianza totale.

PCA: mappe dei fattori delle variabili (variable factor map)



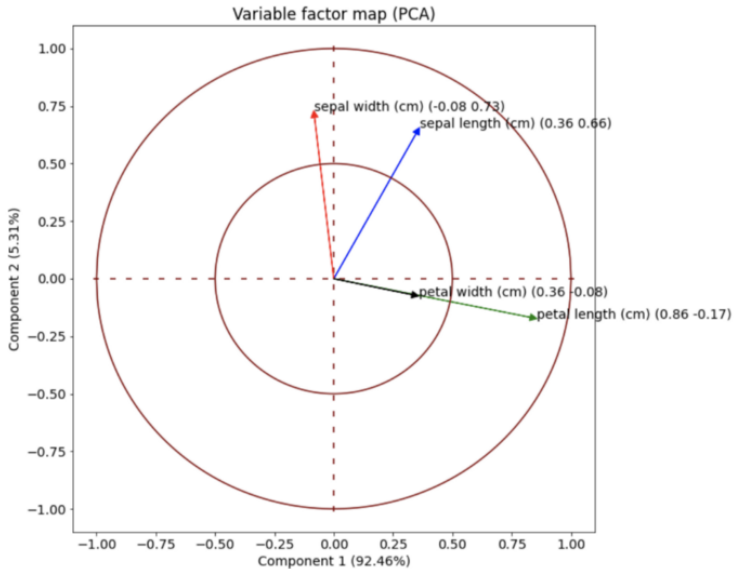
Se il numero di caratteristiche è 2, possiamo inserirle in un grafico 2D e visualizzare come le diverse caratteristiche tengono conto di ogni nuovo componente.

Nell'immagine successiva, quattro caratteristiche (larghezza del sepalo, lunghezza del sepalo, larghezza del petalo, lunghezza del petalo) dal set di dati dell'iride sono compresse in due.

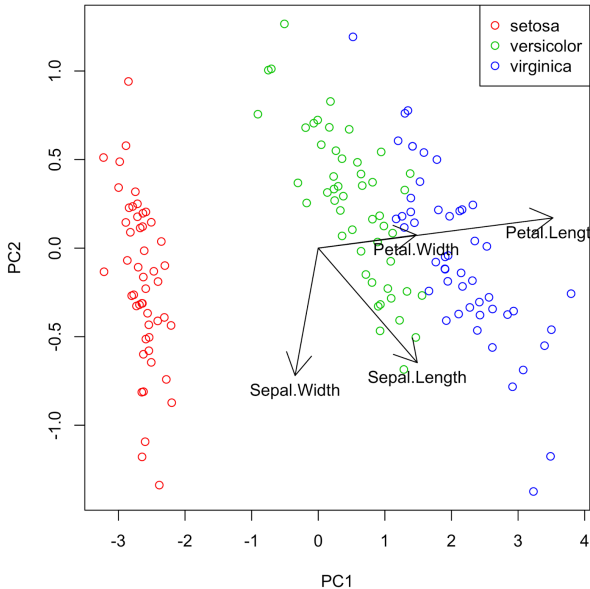
Sono la componente 1 (asse y) e la componente 2 (asse x), che mantengono il 97,77% delle informazioni del set di dati originale.

I quattro vettori rappresentano come queste 2 componenti spiegano le quattro caratteristiche originali.

PCA: mappe dei fattori delle variabili (variable factor map)



PCA: mappe dei fattori delle variabili (variable factor map)



Esercizi

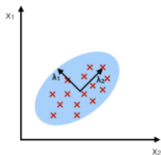
- Lanciamo PCA sui dataset di scikit-learn ed estraiamo la importanza delle feature e la loro composizione (data una componente principale dire i pesi delle componenti originarie per quella specifica componente).

LDA: linear discriminant analysis

Sia LDA che PCA si basano su trasformazioni lineari e mirano a **massimizzare la varianza** in una dimensione inferiore. Tuttavia, a differenza di PCA, LDA trova i discriminanti lineari per **massimizzare la varianza tra le diverse categorie** riducendo al **minimo la varianza all'interno della classe**.

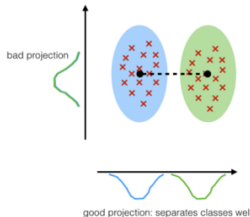
PCA:

component axes that maximize the variance



LDA:

maximizing the component axes for class-separation



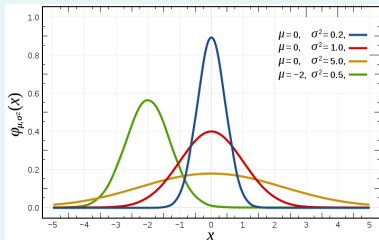
Parleremo più approfonditamente della LAD nel corso di teoria, in quanto la stessa tecnica può essere usata per addestrare un modello classificatore.

Distribuzione Gaussiana o normale

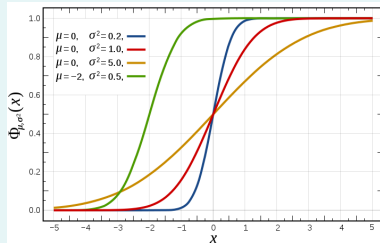
$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

dove μ è la media e σ è la deviazione standard.

Probability density function



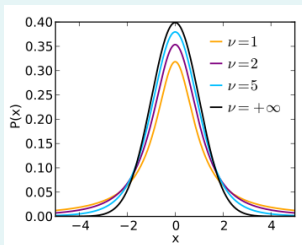
Cumulative function



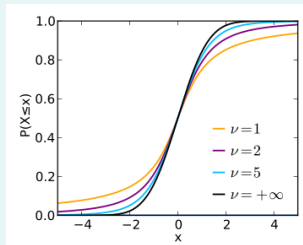
Students' t -distribution

E' utilizzata per stimare la media di una distribuzione gaussiana in situazioni dove la grandezza del campione (sample size) è piccola e la deviazione standard della popolazione non è conosciuta.

Probability density function



Cumulative function



Students' t -distribution

Data una variabile aleatoria X . Sia n la dimensione del campione. Approssimiamo la media e la varianza (corretta seconda Bessel, ovvero usando $n - 1$ campioni invece che n) rispettivamente come

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i$$
$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

Allora, la variabile $\frac{\bar{X}-\mu}{\sigma/\sqrt{n}}$ è una distribuzione normale standard (media 0 e varianza 1) mentre,

$\frac{\bar{X}-\mu}{S/\sqrt{n}}$ è una Students' t -distribution con $n - 1$ gradi di libertà.

La pdf è data da:

$$f(t) = \frac{\Gamma(\frac{n+1}{2})}{\sqrt{n\pi}\Gamma(\frac{n}{2})} \left(1 + \frac{t^2}{n}\right)^{-(n+1)/2} = \frac{1}{\sqrt{n}B(\frac{1}{2}, \frac{n}{2})} \left(1 + \frac{t^2}{n}\right)^{-(n+1)/2}$$

dove Γ e B sono rispettivamente la funzione gamma e la funzione beta.

La **PCA** cerca di catturare il più possibile proprietà globali del dataset. Per questo motivo spesso può portare a dei **problemi di inconsistenza locale**. Punti originariamente distanti vengono rimappati come vicini.

La **t-SNE** (*t*-Distributed Stochastic Neighbor Embedding) è una tecnica di riduzione della dimensionalità che predilige l'informazione locale a quella globale, tale che le relazioni di vicinanza vengano conservate quando si riduce la dimensionalità.

Svantaggi

- è una distorsione non lineare, difficile da interpretare
- una volta calcolata non è facile (ma possibile) mappare un nuovo dato, motivo per cui è spesso utilizzata solo per la visualizzazione dei dati
- le prestazioni dipendono dal dataset di input e da dettagli algoritmici

Stochastic Neighbor Embedding (SNE)

Idea di base: codificare L'Informazione di vicinato originaria come una distribuzione.

Per fare questo, dato un punto, assegniamo una alta probabilità a punti ad esso vicini e una bassa probabilità ai punti ad esso lontani.

A questo però abbiamo ottenute diverse distribuzioni di probabilità, una per ogni punto, che vanno poi confrontate. Per fare ciò utilizziamo la divergenza entropica.

Consideriamo il **vicinato** attorno a un punto $x_i \in \mathbb{R}^d$.

Immaginiamo di avere una **distribuzione gaussiana** centrata in x_i .

Allora la probabilità che x_i scelga un altro punto del data set x_j come suo vicino è proporzionale alle **densità** della gaussiana centrata in esso.

Un punto vicino a x_i avrà una probabilità più alta rispetto ad un punto lontano da x_i .

SNE: distribuzioni di vicinato

Sia d una funzione distanza (tipicamente la distanza euclidea).

La probabilità che x_i scelga x_j come vicino è definita come

$$P_{j|i} = \frac{e^{\frac{-d(x_i, x_j)^2}{2\sigma_i^2}}}{\sum_{k \neq i} e^{\frac{-d(x_i, x_k)^2}{2\sigma_i^2}}}$$

con $P_{i|i} = 0$.

Il parametro σ_i serve a scegliere la **dimensione del vicinato**: per **valori bassi** solo i vicini più prossimi avranno alta probabilità; per **valori alti**, le probabilità sono uniformi.

Scegliamo un specifico valore σ_i differente per ogni dato in base al valore di *perplexità*. **Problema**: $P_{i|j}$ è diverso da $P_{j|i}$. **Soluzione**: passiamo a una distribuzione simmetrica $P_{ij} = \frac{1}{2N}(P_{i|j} + P_{j|i})$.

SNE: perplessità (perplexity)

Sia P_i la distribuzione di probabilità condizionata di x_i su tutti gli altri punti del dataset.

Allora la **perplessità** di tale distribuzione è definita come

$$\text{perplexity}(P_i) = 2^{H(P_i)}$$

con

$$H(P_i) = - \sum_j P_{j|i} \log_2(P_{j|i})$$

La perplessità misura quindi l'effettivo numero di vicino del punto i .

Se P_i è una distribuzione uniforme su k elementi, allora la perplessità è k .
Ad alti valori di perplessità corrispondono alti valori di σ^2 della distribuzione, e viceversa.

SNE: perplessità (perplexity)

L'algoritmo originale esegue una ricerca binaria sul valore di σ_i per trovare la perplessità del punto x_i . Tuttavia, essa è un parametro che spesso viene scelto dall'utente perché differenti valori di perplessità catturano differenti scale.



Obiettivi

Dati i punti $x_1, \dots, x_N \in \mathbb{R}^D$, definiamo le distribuzioni P_{ij} con l'obiettivo di trovare un buon *embedding* (incastro) $y_1, \dots, y_N \in \mathbb{R}^d$ per qualche $d < D$ (tipicamente $d = 2$ o 3).

Come misuriamo la qualità di un embedding?

Per i punti $y_1, \dots, y_N \in \mathbb{R}^d$ possiamo definire una distribuzione Q come

$$Q_{ij} = \frac{e^{-d(y_i, y_j)^2}}{\sum_k \sum_{l \neq k} e^{-d(y_l, y_k)^2}}$$

Cerchiamo quindi di **ottimizzare** Q in modo che sia più simile a P . Per farlo **minimizziamo** la divergenza entropica tra P e Q . Ovvero, i parametri che stiamo ottimizzando, tale che Q sia più possibile simile a P , sono i nostri $y_1, \dots, y_N \in \mathbb{R}^d$.

Sia $L(Q) = KL(P||Q)$.

Notiamo che $KL(P||Q) = \sum_{ij} P_{ij} \log(\frac{P_{ij}}{Q_{ij}}) = -\sum_{ij} P_{ij} \log(Q_{ij}) + \text{cost.}$

Si può dimostrare che $\frac{\partial L}{\partial y_i} = \sum_j (P_{ij} - Q_{ij})(y_i - y_j)$, e che quindi possiamo ottimizzare facendo dei restart multipli.

Crowding problem

Dato un punto x_i avente k vicini nella spazio originario, quando passiamo ad un spazio di dimensione inferiore non riusciamo ad accomodare tutti i suoi vicini.

Se in 2D ho k vicini a distanza 1, in 1D a distanza 1 ne potrò avere al più 2.

Per alleviare i problemi dovuti al crowding problem, utilizziamo una distribuzione con code più lunghe ... la t-distribution è l'ideale.

Possiamo definire la t-distribution come $p(x) \simeq (1 + \frac{x^2}{v})^{\frac{-(v+1)}{2}}$.

Per $v = 1$, abbiamo $p(x) \simeq \frac{1}{1+x^2}$.

Definiamo, quindi, Q_{ij} come

$$Q_{ij} = \frac{(1 - d(y_i, y_j)^2)^{-1}}{\sum_k \sum_{l \neq k} (1 + d(y_k, y_l)^2)^{-1}}$$

Algorithm 1: Simple version of t-Distributed Stochastic Neighbor Embedding.

Data: data set $\mathcal{X} = \{x_1, x_2, \dots, x_n\}$,
 cost function parameters: perplexity $Perp$,
 optimization parameters: number of iterations T , learning rate η , momentum $\alpha(t)$.
Result: low-dimensional data representation $\mathcal{Y}^{(T)} = \{y_1, y_2, \dots, y_n\}$.

begin
 compute pairwise affinities $p_{j|i}$ with perplexity $Perp$ (using Equation 1)
 set $p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}$
 sample initial solution $\mathcal{Y}^{(0)} = \{y_1, y_2, \dots, y_n\}$ from $\mathcal{N}(0, 10^{-4}I)$
 for $t=1$ **to** T **do**
 compute low-dimensional affinities q_{ij} (using Equation 4)
 compute gradient $\frac{\delta C}{\delta \mathcal{Y}}$ (using Equation 5)
 set $\mathcal{Y}^{(t)} = \mathcal{Y}^{(t-1)} + \eta \frac{\delta C}{\delta \mathcal{Y}} + \alpha(t) (\mathcal{Y}^{(t-1)} - \mathcal{Y}^{(t-2)})$
 end
end

UMAP sta per Uniform Manifold Approximation and Projection for Dimension Reduction.

E' una tecnica di riduzione della dimensionalità che si avvicina a t-SNE in quanto entrambe sono basate sulla costruzione di un grafo di similarità tra i punti.

E' usata sia per la visualizzazione dei dati ma anche come passo di preprocessing per il clustering. Infatti, a differenza di t-SNE, è facile ottenere la sua funzione inversa.

La prima versione è stata introdotto nel 2018 e successivamente revisionata nell'articolo del 2020

- articolo 2018 <https://arxiv.org/pdf/1802.03426v1.pdf>
- articolo 2020 <https://arxiv.org/pdf/1802.03426v3.pdf>

La documentazione ufficiale è disponibile all'indirizzo <https://umap-learn.readthedocs.io/en/latest/>.

UMAP sta per Uniform Manifold Approximation and Projection for Dimension Reduction.

Come input si ha una matrice di distanze $N \times N$ costruita ad esempio a partire da N punti in uno spazio Euclideo $\mathbb{R}^m$.

Ha due parametri:

- numero di vicini k da considerare per ogni punto
- numero di dimensioni d dell'embedding di output

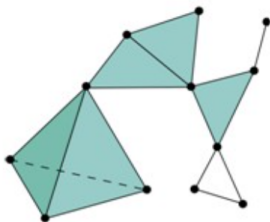
2 passi principali

- **semplificazione** della topologia
- **riduzione** della dimensionalità

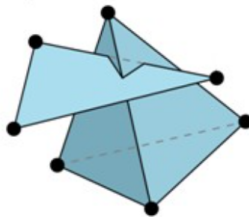
UMAP - semplificazione della topologia

Un **simpleso n -dimensionale** (simplex) è un poligono col minor numero di vertici.

Un **complesso simpliciale** $\mathcal{K}$ è una collezione finita di insiemi di semplici tale che: (i) ogni faccia di un simpleso di $\mathcal{K}$ è anch'essa in $\mathcal{K}$; (ii) l'intersezione di qualunque due semplici di $\mathcal{K}$ è una faccia di ognuno di loro.



Simplicial complex



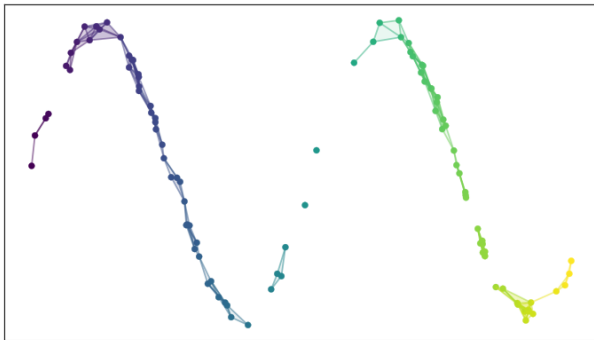
Invalid Simplicial complex

Con la metodologia fuzzy mappiamo (normalizziamo) gli insiemi di semplici nell'intervallo $[0, 1]$.

UMAP - semplificazione della topologia

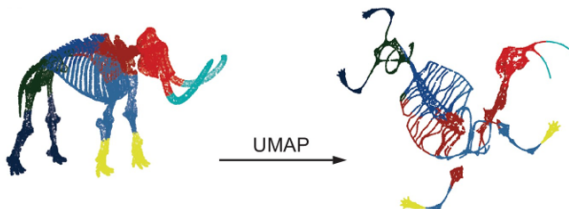
Passi:

- per ogni punto $i = 1, \dots, N$ costruisce uno **spazio** *quasi metrico* $\mathcal{M}_i$ locale a i normalizzando le distanze rispetto ai k punti più vicini ad i .
- **distilla** il contenuto topologico e geometrico di ogni $\mathcal{M}_i$ in un insieme simpliciale fuzzy F_i
- l'**unione** fuzzy $\bigcup_i F_i$ è una rappresentazione topologica globale.

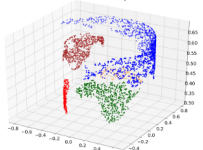


Passi:

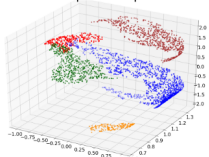
- inizializza una nuvola Z di N punti nello spazio euclideo $\mathbb{R}^d$
- usa **fuzzy set cross-entropy** per misurare la distanza tra le rappresentazioni fuzzy simpliciali di Z e dei punti di input
- muovi i punti in Z fino a minimizzare tale distanza



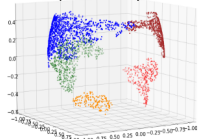
Swiss Roll: 3023 points



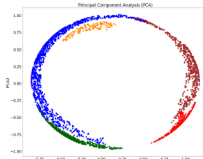
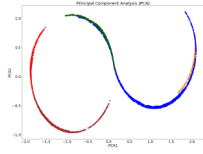
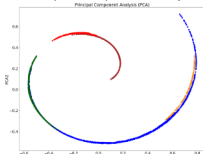
S-shape: 3023 points



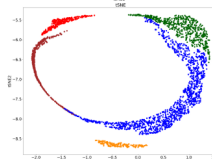
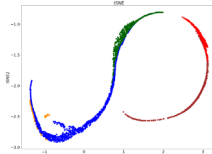
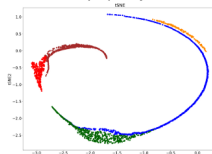
Sphere: 3023 points



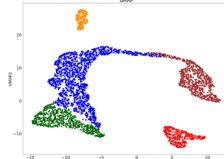
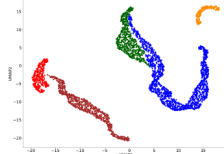
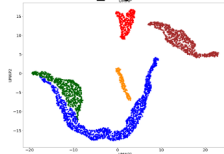
Principal Component Analysis



tSNE: perplexity = 2000



UMAP: n_neighbor = 2000



Tecniche più recenti sono **TriMAP** e **PacMAP**.

Original Data



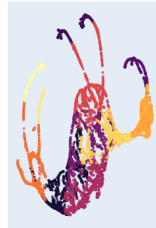
t-SNE - 69.27s



UMAP - 24.12s



TriMAP - 12.56s



PaCMap - 8.42s



Esercizi

- lanciamo i metodi di riduzione della dimensionalità disponibili su scikit-learn, più UMAP, sui dataset di scikit-learn...analizzare le differenze

Self-organizing maps

Una mappa auto-organizzante (SOM) è una tecnica di apprendimento automatico **non supervisionata** utilizzata per produrre una rappresentazione a bassa dimensione (tipicamente bidimensionale) di un set di dati di dimensione superiore **preservando la struttura topologica dei dati**.

Una SOM è un tipo di rete neurale artificiale che, a differenza delle classiche reti neurali, viene addestrata utilizzando l'**apprendimento competitivo** piuttosto che l'apprendimento con correzione degli errori (ad esempio, backpropagation con discesa del gradiente).

La SOM è stata introdotta da Kohonen negli anni '80 e quindi è talvolta chiamata mappa Kohonen o rete Kohonen. La mappa di Kohonen è un'astrazione computazionalmente conveniente costruita su **modelli biologici** di sistemi neurali degli anni '70 e modelli di morfogenesi risalenti a Turing negli anni '50.

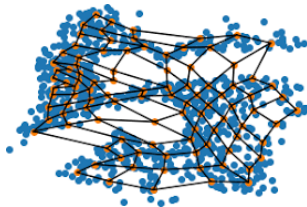
Self-organizing maps

Le SOM hanno due **livelli**, il primo è il livello di input e il secondo è il livello di output o la mappa delle caratteristiche.

A differenza di altri tipi di ANN, SOM non ha una funzione di attivazione nei neuroni, passiamo direttamente i pesi al livello di output senza fare nulla.

A ogni neurone in una SOM viene assegnato un **vettore peso** con la stessa dimensionalità d dello spazio di input.

Quindi, viene utilizzato il competitive learning per "imparare" i pesi.



Self-organizing maps

Il competitive learning è composto da **tre processi** principali:

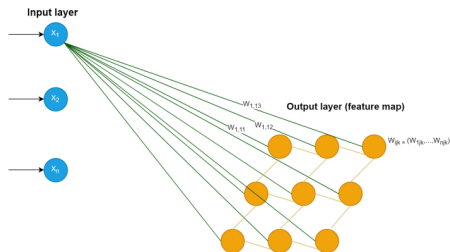
- competizione
- cooperazione
- adattamento

Competizione

In ogni neurone dello strato di output avremo un vettore di dimensione n .

Calcoliamo la distanza tra ciascun neurone (neurone dal livello di output) e i dati di input e il neurone con la **distanza più bassa** sarà il vincitore della competizione.

La metrica euclidea è comunemente usata per misurare la distanza.

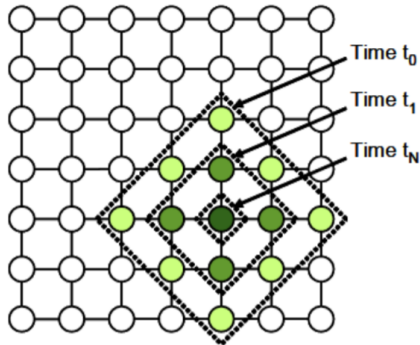


Cooperazione

Aggiungeremo il vettore del neurone vincitore nel processo finale (adattamento) ma non è l'unico, verrà aggiornato anche il suo vicino.

Come scegliamo i vicini?

Per scegliere i vicini usiamo la funzione del **kernel di vicinato**, questa funzione dipende da due fattori: il **tempo** (il tempo aumenta ad ogni nuovo dato di input) e la **distanza** tra il neurone vincitore e l'altro neurone (Quanto dista il neurone dal neurone vincitore).



L'immagine ci mostra come vengono scelti i vicini del neurone vincitore (quello più verde al centro) in base a fattori di distanza e tempo.

Self-organizing maps

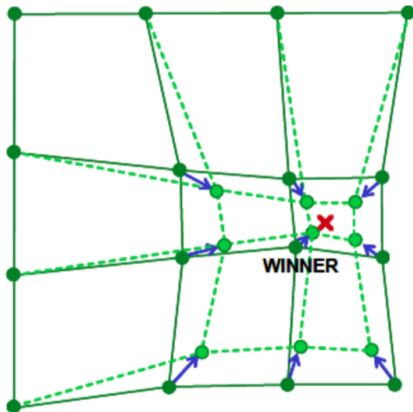
Adattamento

Dopo aver scelto il neurone vincitore e i suoi vicini, calcoliamo l'aggiornamento dei neuroni. Quei neuroni scelti verranno aggiornati ma non la stessa "forza", più la distanza tra il neurone e i dati di input cresce meno forza applichiamo (ovvero meno lo regoliamo).

L'aggiornamento avviene secondo la formula

$$w_k = w_k + \eta(t) \cdot \eta_{ik}(t) \cdot (x^n - w_k)$$

dove η è una funzione a decadimento esponenziale, tipicamente $\eta(t) = \eta_0 e^{-\frac{t}{\tau_1}}$



Dopo il tempo t (infinito positivo), questo tasso di apprendimento convergerà a zero, quindi non avremo aggiornamenti nemmeno per il vincitore del neurone.

Il kernel del vicinato dipende dalla distanza tra il neurone vincitore e l'altro neurone (sono proporzionalmente invertiti: d aumenta e fa diminuire $h(t)$) e dalla dimensione del vicinato che dipende dal tempo (diminuisce mentre aumenta il tempo) e questo rende la funzione del kernel del vicinato monotona decrescente.